プログラミング及び実習2

第 7 回課題（ex07・模擬試験）の解答例

佐野 彰

ex07 解答例: no_if_parity.c（if を使わずに奇数と偶数を扱う）

```c
#include <stdio.h>

int main() {
    int n, x;
    int odd = 0, even_sum = 0;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        odd += x % 2;                   // 奇数なら 1、偶数なら 0 を足す
        even_sum += x * (1 - x % 2);    // 偶数なら x、奇数なら 0 を足す
    }
    printf("odd %d\n", odd);
    printf("even_sum %d\n", even_sum);
    return 0;
}
```

- `x % 2` は奇数なら 1、偶数なら 0。この値をそのまま足せば奇数の個数になる
- `1 - x % 2` は偶数なら 1、奇数なら 0。`x` に掛けると、偶数のときだけ `x` が足される

ex07 解答例: even_sort.c（偶数だけ並び替え）

```c
#include <stdio.h>
#define N 100

int main(void) {
    int n;
    do {
        scanf("%d", &n);
    } while (n < 1);

    int a[N];
    for (int i=0; i<n; i++)
        scanf("%d", &a[i]);

    for (int i=0; i<n-1; i++) {
        if (a[i] % 2 != 0) continue;

        for (int j=i+1; j<n; j++) {
            if (a[j] % 2 != 0) continue;

            if (a[i] > a[j]) {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }

    for (int i=0; i<n; i++)
        printf("%d ", a[i]);
    printf("\n");

    return 0;
}
```

- 第 6 回の並び替えで、**偶数の位置どうしだけ**を比べて入れ替える。奇数の位置は `continue` で飛ばす
- 負の奇数は `a[i] % 2` が `-1` になるので、`== 1` ではなく `!= 0` で調べる

ex07 解答例: neighbor.c（隣接平均で平滑化）

```c
#include <stdio.h>
#define N 100

int main(void) {
    int n;
    do {
        scanf("%d", &n);
    } while (n < 1);

    double a[N], b[N];
    for (int i=0; i<n; i++)
        scanf("%lf", &a[i]);

    if (n == 1) {
        b[0] = a[0];
    } else {
        b[0] = (a[0] + a[1]) / 2.0;
        for (int i=1; i<n-1; i++)
            b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0;
        b[n-1] = (a[n-2] + a[n-1]) / 2.0;
    }

    for (int i=0; i<n; i++)
        printf("%.2f ", b[i]);
    printf("\n");

    return 0;
}
```

- 結果は別の配列 `b` に入れる。`a` を書き換えながら進むと、次の要素の計算に平滑化後の値が混ざる
- 先頭と末尾は 2 つの平均、`n == 1` のときは `a[0]` のまま。`a[-1]` や `a[n]` を読まないようにする

ex07 解答例: duplicate_once.c（2 回以上出現する値）

```c
#include <stdio.h>
#define N 100
#define VALUES 11

int main(void) {
    int n;

    while (1) {
        scanf("%d", &n);
        if (n < 2) break;

        int a[N];
        int cnt[VALUES] = {0};

        for (int i=0; i<n; i++) {
            do {
                scanf("%d", &a[i]);
            } while (a[i] < 0 || a[i] > 10);
        }

        for (int i=0; i<n; i++)
            cnt[a[i]]++;

        for (int i=0; i<VALUES; i++) {
            if (cnt[i] >= 2)
                printf("%d ", i);
        }
        printf("\n");
    }

    return 0;
}
```

- 値が 0〜10 に限られるので、値ごとの出現回数を `cnt[値]` に数える。小さい順に調べれば、そのまま昇順で出る
- 範囲外の値は `do ～ while` で読み直して捨てる。`cnt` は繰り返しのたびに 0 に戻す

ex07 解答例: why_output.c（出力の理由を説明する）

```
    printf("%.2f\n", (double)a / b);
    // 出力: 3.50。a を double にキャストしてから割るので、b も double に変換されて実数の割り算になる
    printf("%c %d\n", c, c);
    // 出力: C 67。文字は整数（ASCII コード）なので 'A'（65）+ 2 = 67 は 'C'。
    //       %c は文字、%d はコードの値を表示する
    printf("%d\n", big + 1);
    // 出力: -2147483648。int は 4 バイトで、表せる範囲は -2147483648〜2147483647。
    //       最大値に 1 を足すと範囲を超えて、最小値 -2147483648 になる
    printf("%u\n", u);
    // 出力: 4294967295。符号なし整数に -1 を代入すると、2^32 - 1（unsigned int の最大値）として扱われる
```

- 出力の値だけでなく、**型（int・double・char・unsigned int）と値の範囲**から理由を書く
- `(double)(a / b)` と書くと、先に int の割り算で 3 になってから変換されるので `3.00` になる

ex07 解答例: fill_increasing.c（空欄を埋める・増え続ける区間）

```
    for (int i = 1; i < n; i++) {
        // ここから
        if (a[i] <= a[i - 1]) {
            start = i;
        }
        if (i - start + 1 > best_len) {
            best_len = i - start + 1;
            best_start = start;
        }
        // ここまで
    }
```

- 直前の値より大きくなければ（**等しいときも**）、区間が途切れるので `i` から新しい区間を始める
- いまの区間の長さは `i - start + 1`。これまでの最長より**長いときだけ**更新するので、同じ長さなら最初の区間が残る

ex07 解答例: digit_shift.c（数字だけシーザー暗号）

```c
#include <stdio.h>
#define MAX_CHARS 100

int main(void) {
    char c;
    int i = 0;

    while (i < MAX_CHARS) {
        scanf("%c", &c);
        if (c == ' ') break;

        if (c >= '0' && c <= '9') {
            if (c == '9') {
                c = '0';
            } else {
                c++;
            }
        }
        printf("%c", c);
        i++;
    }

    printf("\n");
    return 0;
}
```

- 文字は整数なので、`c >= '0' && c <= '9'` で数字かどうかを調べ、`c++` で次の文字にできる。`'9'` だけ `'0'` に戻す
- 空白を読んだら `break` で抜ける。空白は `printf()` より前で抜けるので出力されない
- 読んだ文字数を `i` で数え、100 文字で止める（101 文字目以降は読まない）

ex07 解答例: fixbugs.c（誤りを直す・最大値・最小値・平均値）

```c
    int max = a[0];
    int min = a[0];  // 修正
    int sum = 0;
    for (i = 0; i < n; i++) {  // 修正
        if (a[i] > max) {
            max = a[i];
        }
        if (a[i] < min) {
            min = a[i];
        }
        sum += a[i];  // 修正
    }
    double average = (double)sum / n;  // 修正
```

1. `min = 0` だと、正の値しかないとき最小値が 0 のままになる → `a[0]` から始める
2. `i < n - 1` だと最後の要素を見ない → `i < n`
3. `sum = a[i]` は足していない（最後の値で上書き） → `sum += a[i]`
4. `sum / n` は int どうしの割り算で小数部が消える → `(double)sum / n`